Implementation and Training of YOLOv1 for Object Detection on VOC Dataset

Thadar Soe[1]
[1]Pusan National University

Object detection is a key task in computer vision that involves identifying and locating multiple objects within an image. Unlike image classification, which predicts only one label per image, object detection must determine both the category and the position of each object. The You Only Look Once (YOLO) [5], first introduced as YOLOv1, performs object detection in a single forward pass of a convolutional neural network. YOLO is a major advancement in real-time object detection, enabling high-speed processing. The model divides the image into a grid and predicts bounding boxes and class probabilities directly for each grid cell, which distinguishes it from methods like Faster R-CNN [6] and RetinaNet [3], which use region proposals and anchors for detection. YOLO's ability to make predictions in a single pass is one of its key advantages, making it highly efficient for real-time applications.

This report summarizes the implementation and training results for the YOLOv1 object detection model using PyTorch[7]. The main goals of this Lab2 assignments were to correctly build the network architecture, implement the YOLO loss function, and verify that the model performs close to the refer- ence pretrained version (around 0.685 mAP) by training the model on the VOC[1] 2007+2012 dataset to reproduce and analyze detection performance.

The trained YOLO network achieved the following mean Average Precision (mAP) scores on the validation dataset:

Category	AP (%)
Aeroplane	63.10
Bicycle	65.38
Bus	68.72
Car	68.68
Cat	77.10
Dog	73.72
Mean Average Precision (mAP)	**69.45**

1. Implementation

1.1. Dataset and Target Generation

The training images were collected from the combined PASCAL VOC 2007 and VOC 2012 datasets[1]. Each image was resized and divided into a 7×7 grid so that the model could predict objects in different regions of the image. For every grid cell, the network predicted B=2 bounding boxes to indicate the object's position and size within the photo, along with C=20 class probabilities. This made the final output tensor to have the shape of $7\times7\times30$. The `generate_target` function was used to prepare the target data by normalizing the bounding box coordinates and encoding the ground truth information into this fixed structure. This process allowed the model to learn how to locate and classify objects accurately during training.

1.2. Network Architecture

The implemented YOLO network was designed according to the given assignment specification, consisting of 22 convolutional layers, 4 max-pooling layers, and 2 fully connected layers. The convolutional layers were responsible for extracting spatial and semantic features from the input image, while the pooling layers gradually reduced the spatial resolution, allowing the network to capture more global context. The final fully connected layers combined these extracted features to produce predictions for each grid cell in the image.

A LeakyReLU activation function with a negative slope of 0.1 was applied after every convolutional layer except the last one, which used a linear output for prediction. To downsample the feature maps, the first and twentieth convolutional layers were set with a stride of 2, effectively halving the spatial dimensions of the feature maps. The final output of the network was reshaped into a tensor of size $(7,7,30)$, representing 7-by-7 grid cells, each containing two bounding boxes and twenty class probabilities.

For training initialization, pretrained weights from `pre_train.pt` were loaded to speed up convergence and improve stability. All layers were initialized using the He normal initialization method[2], which is suitable for networks using ReLU-based activation functions. This configuration allowed the model to learn both localization and classification efficiently during the training process.

1.3. Loss Function

The loss function (`yoloLoss`) was implemented according to the YOLOv1 paper[5], combining several components to jointly optimize classification, localization, and confidence prediction. The final loss was computed as a weighted sum of all partial errors, with $\lambda_{coord} = 5$ for coordinate terms and $\lambda_{noobj} = 0.5$ for cells without objects. The implementation[4] was divided into three main parts:

- **Classification loss:** Implemented in `compute_prob_error()`, this part measures the mean-squared error between predicted and target class probabilities, but only for grid cells that contain at least one object (`target[:,:,:,4] > 0`). It extracts the last 20 elements of each cell's output vector, corresponding to class probabilities, and computes their difference using `F.mse_loss()`.
- **No-object loss:** Implemented in `not_contain_obj_error()`, this part penalizes confidence scores for cells that do not contain any object (`target[:,:,:,4] == 0`). Both bounding box predictors per cell are considered, and their confidence scores (indices 4 and 9 in the 30-dimensional output) are pushed toward zero. This loss is weighted by λ_{noobj} to reduce its influence.
- **Localization and object loss:** Implemented in `contain_obj_error()`, this part handles both localization and confidence prediction for cells that contain objects. For each object, two bounding boxes are predicted, and the "responsible" one is chosen based on the higher Intersection-over-Union (IoU) with the ground truth box. Using this selection, the network computes coordinate loss for (x,y) positions, dimension loss for $\sqrt{w}$ and $\sqrt{h}$ to reduce the effect of large boxes, object confidence loss using the IoU as the target value. The other bounding box in the same cell is treated as "not responsible" and contributes only to the no-object loss.

2. Training Setup & Results

The model was trained using SGD with learning rate 1×10^{-4}, momentum 0.9, and weight decay 5×10^{-4}. The batch size was 10 and the training ran for 15 epochs. The network was trained using a pretrained initialization achieving stable convergence without exploding gradients.

The detailed evaluation results across categories show that the model performed consistently well among different object types. High accuracy was observed for animal classes such as *cat* (77.10%) and *dog* (73.72%), which typically have distinctive shapes and textures that make them easier for the network to recognize. Vehicle classes such as *bus* (68.72%) and *car* (68.68%) also achieved solid results, indicating that the model successfully learned features for rigid and structured objects. However, relatively lower performance in categories like *aeroplane* (63.10%) and *bicycle* (65.38%) suggests that the model had more difficulty with elongated or complex shapes that vary greatly in orientation and scale.

According to the official guideline, the pretrained reference model achieves approximately **0.685 mAP**, and any significant deviation below this value indicates an incorrect implementation. The reproduced model in this experiment achieved **0.6945 mAP**, slightly higher than the reference, confirming that the architecture, loss function, and training procedure were correctly implemented. The final mAP of **69.45%** closely matches the reference pretrained performance (0.685 mAP), confirming that the network and loss implementations were correct. Qualitative inspection of visualization results also showed accurate localization and class prediction on most test images.

3. Conclusion

Through this experiment, it was able to learn the practical implementation of the YOLOv1 architecture[4] and loss function from scratch using PyTorch[7]. The model achieved a mean Average Precision of **69.45%**, which is consistent with and slightly higher than the reference pretrained performance (**68.5%**). This verifies the correctness of the implementation and demonstrates stable training and convergence. The obtained results also highlight YOLO[5]'s ability to balance detection accuracy and computational efficiency, achieving reliable real-time object detection performance.

4. References

[1] Mark Everingham, Luc Van Gool, Christopher K. I. Williams, John Winn, and Andrew Zisserman. The pascal visual object classes (voc) challenge. volume 88, pages 303–338, 2010. URL https://www.pascal-network.org/challenges/VOC/.

[2] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. 2015. URL https://arxiv.org/abs/1502.01852.

[3] Tsung-Yi Lin, Priya Goyal, Ross B. Girshick, Kaiming He, and Piotr Dollar. Focal loss for dense object detection. 2017. URL https://arxiv.org/abs/1708.02002.

[4] Joseph Redmon. Yolo: Real-time object detection, 2016. https://pjreddie.com/darknet/yolo/.

[5] Joseph Redmon, Santosh Divvala, Ross B. Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. *CVPR*, 2016. URL https://arxiv.org/abs/1506.02640.

[6] Shaoqing Ren, Kaiming He, Ross B. Girshick, and Jian Sun. Faster r-cnn: Towards real-time object detection with region proposal networks. 2015. URL https://arxiv.org/abs/1506.01497.

[7] PyTorch Team. Pytorch: An imperative style, high-performance deep learning library, 2019. https://pytorch.org/.